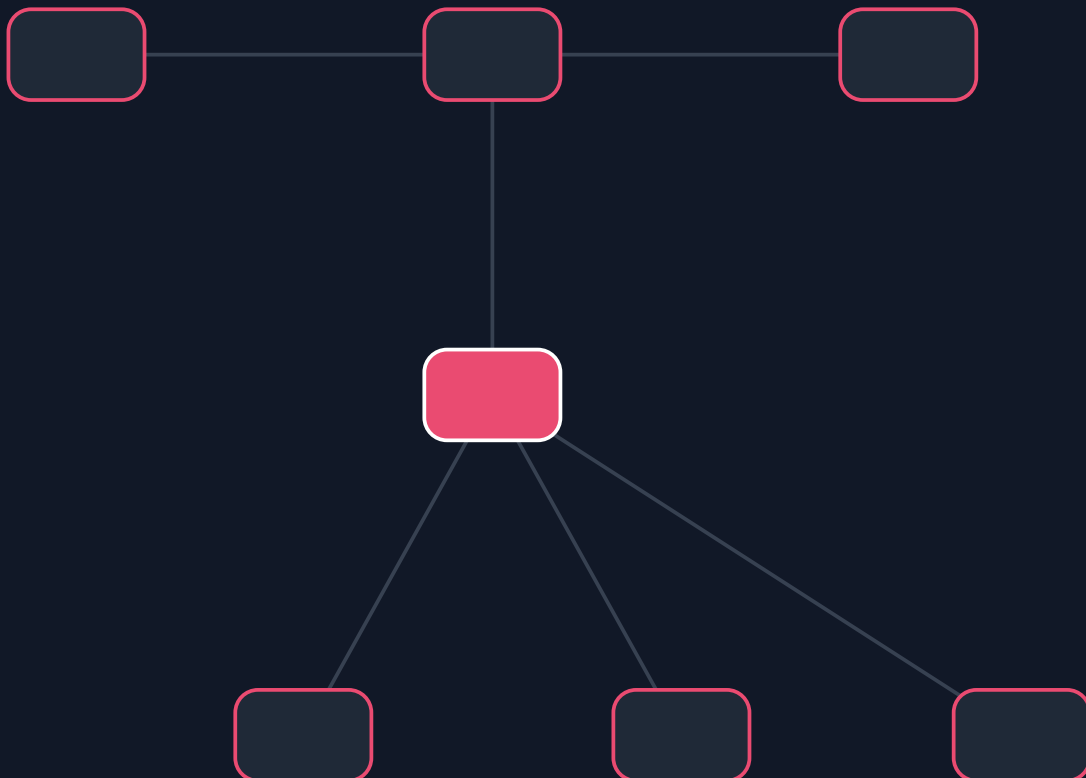


Building Your First AI Agent with n8n

Smart Study Assistant: Chat Trigger, AI Agent, OpenRouter, Memory and Tools, explained node by node



Read this manual completely before the lecture.

Complete Section 9 (Pre-Lab Checklist) and bring your OpenRouter API key.

EshaBuilds

Contents

Contents	2
1. Introduction and Learning Objectives	4
1.1 By the end of the lab you will be able to	4
1.2 What you need	4
2. Core Concepts	5
2.1 Chatbot vs. AI agent	5
2.2 The ReAct loop	5
2.3 How n8n represents an agent	5
2.4 The workflow you will build	6
3. Setting Up n8n	7
3.1 Option A: n8n Cloud	7
3.2 Option B: Self-host on your laptop	7
3.3 Getting to know the editor	7
4. Getting Your OpenRouter API Key	9
4.1 What is OpenRouter?	9
4.2 Step-by-step: create the key	9
4.3 Free model limits (important)	9
4.4 Choosing a free model that supports tools	9
5. Node-by-Node Reference	11
5.1 When chat message received (Chat Trigger)	11
5.2 AI Agent	11
5.3 OpenRouter Chat Model	12
5.4 Simple Memory	13
5.5 Tool: Calculator	14
5.6 Tool: Wikipedia	14
5.7 Tool: HTTP Request (live weather)	14
6. Building the Workflow: Step by Step	16
Stage 1: A plain chatbot	16
Stage 2: Add memory	16
Stage 3: Add the system message	16
Stage 4: Add tools one at a time	16



Stage 5: Test the agent's decisions 16

6.1 Reading the logs 17

7. Troubleshooting 18

8. Glossary 18

9. Pre-Lab Checklist 20

9.1 Self-check questions 20

9.2 Extension challenge (optional) 20

1. Introduction and Learning Objectives

In this lab you will build a **Smart Study Assistant**: a chatbot that can think, remember the conversation, and decide on its own when to use a calculator, look something up on Wikipedia, or fetch live weather data from the internet. You will build it visually in **n8n** without writing traditional code.

This manual is your **pre-lecture reading**. The lecture assumes you already know what each node does and have your accounts and API key ready. Class time will be spent building and experimenting, not on setup.

1.1 By the end of the lab you will be able to

- Explain the difference between a normal chatbot and an **AI agent**.
- Describe the ReAct loop (Think, Act, Observe) that agents run internally.
- Create an OpenRouter account and API key, and connect it to n8n as a credential.
- Configure the Chat Trigger, AI Agent, OpenRouter Chat Model, Simple Memory and Tool nodes.
- Read the agent's execution log to see which tool it chose and why.
- Debug the most common errors (401, 429, tool-calling failures, missing memory).

1.2 What you need

Item	Details
Laptop	Windows, macOS or Linux with a modern browser (Chrome, Edge or Firefox).
Internet	Required. The agent calls OpenRouter and public APIs online.
n8n	n8n Cloud (free trial) or a local install. See Section 3.
OpenRouter account	Free. No credit card needed. See Section 4.
Time	About 45 to 60 minutes of reading and setup before class.

2. Core Concepts

2.1 Chatbot vs. AI agent

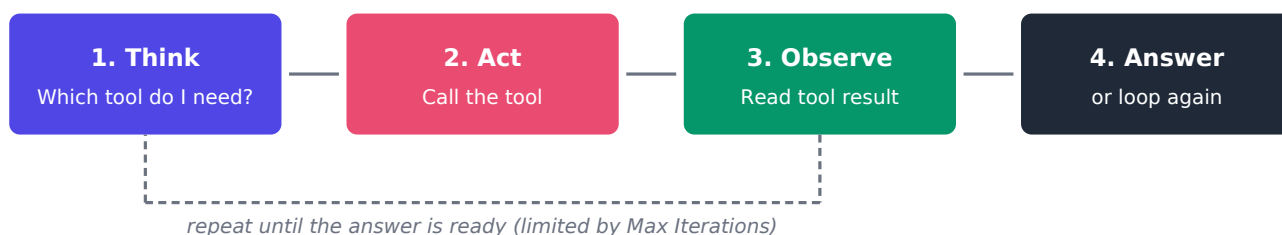
A plain chatbot sends your message to a Large Language Model (LLM) and returns whatever text the model generates. It can only use what it learned during training, so it cannot check today's weather and often makes arithmetic mistakes.

An **AI agent** is an LLM that has been given **tools** and the freedom to **decide** when to use them. When you ask a question, the agent reasons about what it needs, calls a tool, reads the result, and then answers. The key idea: *you do not hard-code which tool runs; the model chooses*.

	Chatbot	AI Agent
Knowledge	Training data only	Training data + live tools
Decisions	Only generates text	Chooses which tool to call and when
Actions	None	Can calculate, search, call APIs, save data
Memory	Often none	Keeps recent conversation with a memory node

2.2 The ReAct loop

Most agents, including n8n's AI Agent node, follow a pattern called **ReAct** (Reason + Act). The model repeats this loop until it is confident it can answer:



Example: for "What is 18% of 3,450 and who invented percentages?" the agent may call Calculator first, then Wikipedia, then combine both results into one answer. That is two loop iterations.

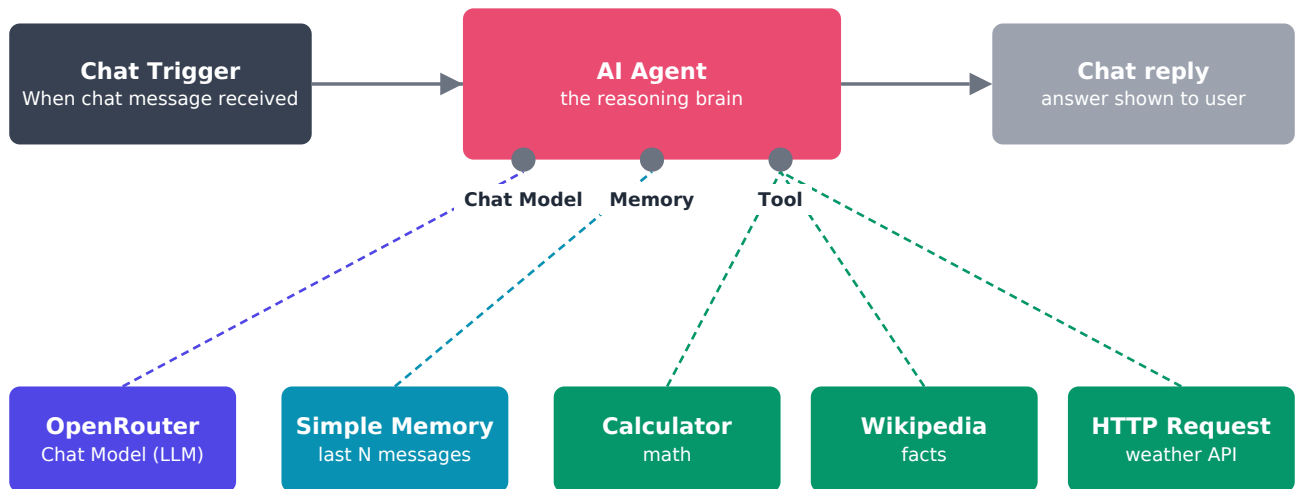
2.3 How n8n represents an agent

n8n workflows are made of **nodes** joined by connections. For AI, there are two kinds:

- **Root nodes** sit in the main flow and process data, for example the **AI Agent**.
- **Sub-nodes** plug into small ports underneath a root node and give it a capability: a **Chat Model** (the brain), a **Memory**, or **Tools**. A sub-node never runs on its own; it only works when attached to a root node.

2.4 The workflow you will build

Solid arrows = data flow (main connections). Dashed lines = sub-nodes plugged into the agent's ports.



Node	Type	Job in one line
When chat message received	Trigger (root)	Starts the workflow when a user sends a chat message.
AI Agent	Root node	Runs the ReAct loop and decides which tool to use.
OpenRouter Chat Model	Sub-node (Model)	The LLM that does the thinking.
Simple Memory	Sub-node (Memory)	Remembers the last few messages of the chat.
Calculator	Sub-node (Tool)	Performs exact math.
Wikipedia	Sub-node (Tool)	Looks up factual information.
HTTP Request	Sub-node (Tool)	Calls the free Open-Meteo API for live weather.

3. Setting Up n8n

Choose **one** option. Option A is the fastest. Option B is free forever and runs on your own machine.

3.1 Option A: n8n Cloud

1. Go to **n8n.io** and click **Get started**.
2. Sign up with your email. You get a free trial with no card required at the time of writing.
3. Verify your email and open your workspace. You will land on the **Overview** page.
4. Click **Create Workflow**. You now have an empty canvas.

Note

The Cloud trial is time-limited. If it expires before the lecture, use Option B.

3.2 Option B: Self-host on your laptop

B1. Using Node.js (simplest)

Install the LTS version of **Node.js** from nodejs.org, then open a terminal and run:

```
npx n8n
```

Wait for the message saying the editor is accessible, then open <http://localhost:5678> in your browser and create the owner account (this account is local to your laptop).

B2. Using Docker

If you have Docker Desktop installed:

```
docker volume create n8n_data
docker run -it --rm --name n8n -p 5678:5678 \
  -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n
```

Then open <http://localhost:5678>. The volume keeps your workflows even after the container stops.

Tip

Keep the terminal window open while you work. Closing it stops n8n.

3.3 Getting to know the editor

Area	What it is for
Canvas	The large grid where you place and connect nodes.
+ button / Tab key	Opens the node panel to search and add nodes.



Area	What it is for
Node panel (double-click a node)	Shows the node's parameters on the left/middle and its input/output data.
Open chat button	Appears at the bottom of the canvas when a Chat Trigger exists. Used to test your agent.
Logs panel	Shows each step the agent took, including tool calls. Your main debugging tool.
Save	Save often (Ctrl/Cmd + S). Unsaved changes are lost on refresh.

4. Getting Your OpenRouter API Key

4.1 What is OpenRouter?

OpenRouter is a single gateway to hundreds of LLMs from many companies (Meta, Google, Mistral, DeepSeek, Qwen, OpenAI, Anthropic and others). With one API key you can switch models with one click, which makes it ideal for learning. Many models have a **free variant** whose ID ends in `:free`.

4.2 Step-by-step: create the key

1. Open **openrouter.ai** and click **Sign up**. Use Google, GitHub, or email.
2. Verify your email if asked. No credit card is needed for free models.
3. Click your profile icon (top right) and open **Keys** (also reachable from Settings).
4. Click **Create API Key** (or **Create Key**).
5. Give it a clear name, for example `n8n-lab`. Leave the credit limit empty or set it to 0 so the key can never spend money.
6. Click **Create**. Your key appears once and starts with `sk-or-v1-`.
7. **Copy it immediately** and store it somewhere safe (a password manager or a private note). You will not be able to see it again. If you lose it, delete it and create a new one.

Security rules for API keys

- An API key is like a password. Anyone with it can use your account.
- Never paste it in WhatsApp groups, screenshots, GitHub repos, or shared documents.
- Store it only inside n8n's **Credentials**, never inside a node's text fields.
- If a key leaks, go to the Keys page and **delete** it right away.

4.3 Free model limits (important)

Free models cost nothing per token but are rate-limited **per account**, not per key:

Account status	Requests per minute	Requests per day
Never bought credits (or under \$10 total)	20	50
Bought \$10 or more of credits at any time	20	1,000

One chat message to an agent can use **2 to 4 requests**, because every loop iteration (think, call tool, think again) is a separate model call. 50 requests per day is enough for the lab, but do not waste them on random testing the night before. Limits are published by OpenRouter and can change; check the Limits page in their docs if something looks different.

4.4 Choosing a free model that supports tools

An agent **must** use a model that supports **tool calling** (also called function calling). Not every free model does. To find one:

1. Go to **openrouter.ai/models**.



2. In the filters, set **Prompt pricing** to **FREE** (or search for **:free**).
3. Under **Supported Parameters**, select **tools**.
4. Pick a well-known instruction-tuned model from the results (Llama, Qwen, Mistral, DeepSeek, Gemma families are common). Write down its exact ID, e.g. **provider/model-name:free**.
5. Write down one backup model too. Free capacity changes, and a model may be temporarily unavailable.

Why no specific model name here?

The list of free models changes every few weeks. Filtering by **free + tools** on the day you set up always gives you a working, current choice.

5. Node-by-Node Reference

This section explains every node in the workflow and every parameter you will touch. Read it carefully; the lecture will refer to these parameter names directly.

5.1 When chat message received (Chat Trigger)

When chat message received

Trigger · root node

Starts the workflow every time someone sends a message in the chat window. It provides the user's text (`chatInput`) and a `sessionId` that identifies the conversation.

How to add: On an empty canvas click **Add first step**, search *chat*, choose **On chat message** (node name: *When chat message received*).

Parameter	What it does	Set it to
Make Chat Publicly Available	Creates a public URL so others can chat with your agent without logging into n8n.	Off for the lab. Use the built-in test chat instead.
Mode	<i>Hosted Chat</i> (n8n shows a chat page) or <i>Embedded Chat</i> (you embed a widget on your own site). Visible only when public.	Hosted Chat (if you make it public later).
Authentication	Who can use the public chat: None, Basic Auth, or n8n User Auth.	Leave default for the lab.
Initial Message(s)	Greeting shown when the chat opens (public chat only).	e.g. "Hi! I am your study assistant."
Options → Response Mode	When the reply is sent. Default sends the output of the last node, which is what we want.	Default.

Output fields you should know:

- `chatInput`: the text the user typed.
- `sessionId`: a unique ID per chat window. Memory uses it to keep conversations separate.
- `action`: usually `sendMessage`.

5.2 AI Agent

AI Agent

Root node · reasoning engine

The heart of the workflow. It receives the user's message, sends it to the chat model together with the system message, memory and tool descriptions, then runs the ReAct loop until it has an answer.

How to add: Click the + on the right side of the Chat Trigger, search *AI Agent*. n8n connects it automatically. Underneath the node you will see three ports: **Chat Model*** (required), **Memory** and **Tool**.

Parameter	What it does	Set it to
Source for Prompt (User Message)	Where the user's question comes from. <i>Connected Chat Trigger Node</i> reads <code>chatInput</code> automatically. <i>Define below</i> lets you write your own expression.	Connected Chat Trigger Node
Require Specific Output Format	Forces structured output (e.g. JSON) through an Output Parser sub-node.	Off (plain text answers).
Enable Fallback Model	Adds a second model port used if the main model fails. Useful with free models.	Optional. Great with your backup free model.
Options → System Message	Permanent instructions that define the agent's role, rules and tool usage. This is the most important setting.	See the prompt below.
Options → Max Iterations	Maximum ReAct loops before the agent gives up. Prevents endless loops.	Default 10 is fine. Lower it (e.g. 5) to save free requests.
Options → Return Intermediate Steps	Includes each tool call and result in the output, so you can study the agent's reasoning.	On during the lab (great for learning).
Options → Enable Streaming	Sends the answer word by word to the chat instead of all at once.	Either. On feels more natural.

System message to use in the lab

You are Smart Study Assistant, a friendly tutor for university students.

Rules:

- For ANY arithmetic or percentage, use the Calculator tool. Never do math in your head.
- For facts about people, places, history or science, use the Wikipedia tool first.
- For current weather, use the `get_weather` tool. Convert city names to latitude/longitude yourself before calling it.
- If no tool is needed, answer directly.
- Keep answers short: 2 to 4 sentences unless the student asks for more.
- If a tool fails, say so honestly. Do not invent results.

Why the system message matters

The model decides which tool to use mostly from two things: the system message and each tool's description. Clear, specific instructions lead to reliable tool use. Vague instructions lead to the agent guessing or skipping tools.

5.3 OpenRouter Chat Model

OpenRouter Chat Model

Sub-node · Chat Model port

Connects the agent to an LLM hosted on OpenRouter. This is the agent's brain: it reads the prompt, decides on tool calls and writes the final answer.

How to add: Click the **Chat Model** port under the AI Agent, search *OpenRouter*, select **OpenRouter Chat Model**.

Creating the credential (do this once)

1. In the node, open **Credential to connect with** and choose **Create new credential**.
2. Paste your `sk-or-v1-...` key into **API Key**. Leave the Base URL as the default (`https://openrouter.ai/api/v1`).
3. Click **Save**. A green *Connection tested successfully* message means it works.
4. Rename the credential to something clear like `OpenRouter - Lab`. Every workflow can reuse it.

Parameter / Option	What it does	Set it to
Credential	The saved API key used for every request.	OpenRouter - Lab
Model	Which LLM to use. n8n loads the list live from OpenRouter for your account.	Your chosen free + tools model ID.
Sampling Temperature	Randomness. Low = focused and consistent, high = creative but more mistakes.	0.2 to 0.4 for reliable tool use.
Maximum Number of Tokens	Caps the length of each response.	Leave default, or ~800.
Frequency / Presence Penalty	Reduce repetition / encourage new topics.	Leave at 0.
Response Format	Text or JSON.	Text.
Timeout	Max wait time per request (ms). Free models can be slow.	Raise to 60000 if you see timeouts.
Max Retries	Automatic retries on temporary errors.	2.

Common confusion

An empty or very short model list almost always means the credential is wrong (bad key or copy-paste with extra spaces), not that the node is broken.

5.4 Simple Memory

Simple Memory

Sub-node · Memory port

Stores the recent messages of the conversation inside n8n so the agent can refer back to them. Without memory, every message is treated as a brand-new conversation. (Older n8n versions call this node *Window Buffer Memory*.)

How to add: Click the **Memory** port under the AI Agent and select **Simple Memory**.

Parameter	What it does	Set it to
Session ID	Which conversation this memory belongs to. <i>Connected Chat Trigger Node</i> uses the trigger's <code>sessionId</code> automatically, so each chat window gets its own memory.	Connected Chat Trigger Node
Session Key	Only shown if you choose <i>Define below</i> . An expression such as <code>{{ \$json.sessionId }}</code> .	Not needed.

Parameter	What it does	Set it to
Context Window Length	How many past interactions (user + AI pairs) are sent back to the model each time.	5. Higher = better memory but more tokens per request.

Limitation

Simple Memory lives in n8n's RAM. It is lost when n8n restarts and is not meant for production. Real apps use Postgres, Redis or similar memory nodes, which we will cover later in the course.

5.5 Tool: Calculator

Calculator

Sub-node · Tool port

Lets the agent evaluate math expressions exactly. LLMs predict text, so they are unreliable at arithmetic; the calculator fixes that.

How to add: Click the **Tool** port, search *Calculator*. It has **no parameters**. The agent sends an expression such as `0.18 * 3450` and receives the result.

5.6 Tool: Wikipedia

Wikipedia

Sub-node · Tool port

Searches Wikipedia and returns article summaries. Gives the agent factual, citable knowledge instead of relying only on what it memorised during training.

How to add: Click the **Tool** port again (a port can hold many tools), search *Wikipedia*. No credentials or parameters are required. The agent decides the search term itself.

5.7 Tool: HTTP Request (live weather)

HTTP Request (as a tool)

Sub-node · Tool port

Lets the agent call any web API. We use **Open-Meteo**, a free weather API that needs no key. This teaches the most powerful idea in agent building: any API can become a tool.

How to add: Click the **Tool** port, search *HTTP Request*, choose **HTTP Request Tool**.

Parameter	What it does	Set it to
Name (node title)	Rename the node. The agent sees this name.	<code>get_weather</code>
Description	Tells the agent when and how to use this tool. Critical for correct tool choice.	See text below.
Method	HTTP verb.	GET
URL	The API endpoint.	<code>https://api.open-meteo.com/v1/forecast</code>

Parameter	What it does	Set it to
Send Query Parameters	Adds ?key=value pairs to the URL.	On, using fields below
Query: latitude	Filled in by the AI.	Let the model define it
Query: longitude	Filled in by the AI.	Let the model define it
Query: current	Which values to return.	<code>temperature_2m,wind_speed_10m,relative_humidity_2m</code>

Letting the model fill a value: next to a field, click the small AI/sparkle button (*Let the model define this parameter*). n8n inserts an expression like:

```
{{ $fromAI('latitude', 'Latitude of the city, e.g. 24.86 for Karachi', 'number') }}
```

This tells the agent: "you must supply this value when you call the tool". Use the description text to guide it. Tool description to paste:

```
Gets the CURRENT weather for a location. Use this whenever the user asks about weather, temperature, wind or humidity. Provide the latitude and longitude of the city in decimal degrees.
```

6. Building the Workflow: Step by Step

We build in stages so you can see what each piece adds. After each stage, test in the chat.

Stage 1: A plain chatbot

1. Create a new workflow and name it **Smart Study Assistant**.
2. Add the **When chat message received** trigger.
3. Add an **AI Agent** after it. Keep Source for Prompt as *Connected Chat Trigger Node*.
4. Attach **OpenRouter Chat Model** to the Chat Model port, select your credential and free tool-capable model.
5. Click **Open chat** and type *Hello, who are you?*. You should get a reply.

Experiment

Ask "My name is Ali." then "What is my name?". The agent forgets, because there is no memory yet.

Stage 2: Add memory

1. Attach **Simple Memory** to the Memory port. Session ID: Connected Chat Trigger Node. Context Window Length: 5.
2. Repeat the name test. It should now remember.

Stage 3: Add the system message

1. Open the AI Agent, click **Add Option** → **System Message**, paste the prompt from Section 5.2.
2. Also add **Return Intermediate Steps** and turn it on.

Stage 4: Add tools one at a time

1. Attach **Calculator**. Test: *What is 17% of 2,450?*
2. Attach **Wikipedia**. Test: *Who was Alan Turing?*
3. Attach **HTTP Request Tool** configured as `get_weather`. Test: *What's the weather in Karachi right now?*
4. Save the workflow.

Stage 5: Test the agent's decisions

Test message	Expected tool(s)	What it proves
What is 17% of 2,450?	Calculator	Agent picks the right tool for math.
Who was Alan Turing?	Wikipedia	Agent retrieves facts instead of guessing.
What's the weather in Hyderabad, Sindh?	get_weather	Agent fills API parameters on its own.

Test message	Expected tool(s)	What it proves
Tell me a fun fact about Einstein and compute 1905×3.	Wikipedia + Calculator	Multi-step reasoning (two loops).
What was my first question?	None (memory)	Memory works.
Write me a two-line poem about exams.	None	Agent knows when NOT to use tools.

6.1 Reading the logs

After each message, open the **Logs** panel at the bottom of the editor. Click on the AI Agent run to see the sequence: model call → tool call (with the exact input the model chose) → tool output → model call → final answer. Studying these logs is the fastest way to understand how agents think.

7. Troubleshooting

Symptom / error	Likely cause	Fix
401 Unauthorized / credential test fails	Wrong or incomplete API key; extra space when pasting.	Re-copy the key or create a new one; update the credential.
429 Too Many Requests / rate limit	Hit 20 per minute or 50 per day free limit.	Wait a minute (or until the next day); reduce Max Iterations; use a fallback model.
Error about tools not supported / agent ignores tools	Chosen model does not support tool calling.	Pick a model filtered by tools on OpenRouter.
Model not available / 404 / empty response	Free provider capacity temporarily gone.	Switch to your backup free model.
Timeout	Free model is slow or overloaded.	Raise Timeout option to 60000 ms or change model.
Agent does math wrong	Did not use Calculator.	Strengthen the system message; lower temperature.
Agent forgets earlier messages	No memory attached or Session ID misconfigured.	Attach Simple Memory with Connected Chat Trigger Node.
Weather tool returns error 400	Latitude/longitude missing or not numbers.	Check the \$fromAI fields and the tool description.
Max iterations reached	Agent is looping (often confused by tool results).	Improve tool descriptions; simplify the question.
"Open chat" button missing	No Chat Trigger in the workflow.	Add When chat message received.

8. Glossary

Term	Meaning
LLM	Large Language Model. A model trained to predict text, e.g. Llama, Qwen, GPT, Claude.
Agent	An LLM that can decide to call tools and loop until it completes a task.
Tool	A function the agent can call: calculator, search, API, database, email and so on.
Tool calling	The model's ability to output a structured request to run a tool with specific inputs.
ReAct	Reason + Act. The think, act, observe loop agents use.
System message	Hidden instructions that define the agent's role and rules.
Token	A small chunk of text (roughly 3 to 4 characters). Models are limited and billed by tokens.
Context window	How much text the model can read at once, including memory and tool results.



Term	Meaning
Temperature	Controls randomness of the output.
API key	A secret string that authenticates you to a service.
Credential (n8n)	Where n8n securely stores API keys so nodes can reuse them.
Root node / sub-node	Main-flow node vs. a node that plugs into a root node's port to add capability.
Session ID	Identifier that keeps different users' conversations separate in memory.

9. Pre-Lab Checklist

Tick every item before the lecture. Students who arrive without these will spend the class on setup.

- ☐ I can open n8n (Cloud workspace or `localhost:5678`) and create a new workflow.
- ☐ I have an OpenRouter account and an API key starting with `sk-or-v1-`, stored safely.
- ☐ I created the OpenRouter credential in n8n and the connection test succeeded.
- ☐ I have written down one primary and one backup free model that support **tools**.
- ☐ I can explain the difference between a chatbot and an agent in two sentences.
- ☐ I can name the three ports on the AI Agent node and what plugs into each.
- ☐ I know why Calculator exists even though LLMs "know" math.
- ☐ I have not wasted my 50 free daily requests the night before class.

9.1 Self-check questions

1. What does the Chat Trigger send to the AI Agent, and which field does Simple Memory use?
2. Why must the chosen OpenRouter model support tool calling?
3. What happens if Max Iterations is set to 1 and the question needs two tools?
4. Why do we use `$fromAI()` in the HTTP Request Tool?
5. A single chat message used 3 requests. How many such messages can you send per day on a free account?
6. Where should an API key be stored in n8n, and where should it never appear?

9.2 Extension challenge (optional)

Add a **Google Sheets** tool so that when a student says "*save this summary*", the agent appends the text to a sheet. This turns your assistant from something that *answers* into something that *acts*, which is the real power of agents. We will build this together in class.

Information about third-party services (n8n, OpenRouter, Open-Meteo) reflects their documentation as of September 2026. Interfaces and free-tier limits can change; if something looks different, check the official docs.